

Predlog projekta iz predmeta Sistemi bazirani na znanju

Uroš Radukić SV54/2022

Opis problema

Motivacija

Trenutno razvijam video igru za [Nordeus Challenge](#). U pitanju je strateška igra po potezima (turn-based RPG), gde igrač i neprijatelj naizmenično biraju kako će iskoristiti svoj potez, dok ne ostane samo jedan od njih živ. Jedna od stavki u izazovu koja je poželjna je da neprijateljski AI ne bira nasumično kako će iskoristiti svoj potez, već da na smislen način odabere koji mu je najbolji potez. Veštačka inteligencija neprijatelja u video igrama je jedan od ključnih faktora koji utiču na kvalitet iskustva igrača. To je posebno izraženo u žanru strateških igara po potezima, ponašanje neprijatelja direktno utiče na to koliko je igra zanimljiva, i koliki izazov predstavlja. Tu sam uvideo idealnu priliku da iskoristim Drools i znanje stečeno na predmetu Sistemi bazirani na znanju i uz svoje ekspertsko znanje iz oblasti strateških video igara dizajniram AI za neprijatelje.

Pregled problema

Postojeća rešenja i njihova ograničenja

Konačni automati (FSM) dugo su bili standard u video igrama. Svako stanje definiše ponašanje, a tranzicije reaguju na događaje. Prednost je jednostavnost implementacije, ali mana je što nema nikakvu fleksibilnost, neprijatelj ima fiksni broj stanja i ne može da reaguje na kombinacije uslova koje dizajner nije eksplicitno predvideo. Konkretna primena u turn-based žanru dokumentovana je u radu [Application of the Finite State Machine Method in the Desktop-Based "Heroes Of Dawn" RPG Turn-Based Game](#) (Riyanto et al., 2022), gde FSM upravlja izborom napada neprijatelja na osnovu trenutnog HP-a i mane. Sistem funkcioniše, ali svaki novi tip neprijatelja zahteva ručno pisanje novih stanja i tranzicija.

Stabla ponašanja (Behavior Trees) nude hijerarhijsku kompoziciju akcija i uslova, što ih čini fleksibilnijim od FSM-a. Popularizovana su igrom *Halo 2* (Bungie, 2004), gde je Bungie prvi put u industriji upotrebio BT za upravljanje borbenom AI NPC-eva. Detalji su opisani u GDC prezentaciji [Handling Complexity in the Halo 2 AI](#) (Isla, GDC 2005). Mana je što su i dalje ručno projektovana i skaliranje postaje problem. Istraživanje [Practical and Theoretical Issues of Evolving Behaviour Trees for](#)

a Turn-based Game (Lim et al., 2019) pokazuje da BT za turn-based igre brzo postaju nepregledni i teški za održavanje čim broj akcija poraste.

Pretraga stabla odluka - Monte Carlo Tree Search (MCTS) je algoritam koji simulira hiljade slučajnih ishoda iz trenutnog stanja igre kako bi pronašao statistički najperspektivniji potez, bez potrebe za ručno kodiranim pravilima. Osnova je položena u radu *Monte-Carlo Tree Search: A New Framework for Game AI* (Chaslot et al., AAAI 2008), a algoritam je postigao vrhunske rezultate u igrama poput Shogija i Go-a. Ograničenje je visoka računarska cena po potezu i loša primenljivost na igre sa velikim brojem stanja ili skrivenim informacijama, kao i to što ne postoji čitljivo obrazloženje zašto je određeni potez izabran.

Utility AI dodeljuje numeričke vrednosti mogućim akcijama na osnovu konteksta i bira onu sa najvišim skorom. Najpoznatija primena je serija *The Sims*, gde svaki objekat u svetu objavljuje skup akcija sa procenjenom korisnošću. Arhitektura je detaljno opisana u radu *Artificial Intelligence in The Sims Series* (Bourse, INRIA 2014). Mada fleksibilan, sistem postaje nepredvidiv pri većem broju faktora i teško ga je ispratiti. Promena jedne krive korisnosti može da promeni ponašanje na nepredviđene načine u sasvim drugom delu igre.

Mašinsko učenje i neuronske mreže nude adaptivnost, ali zahtevaju ogromne količine trening podataka i visoku računarsku snagu. Referentni primer je *AlphaStar* (DeepMind, 2019) za StarCraft II, opisan u radu *Grandmaster level in StarCraft II using multi-agent reinforcement learning* (Vinyals et al., Nature 2019). Sistem je dostigao nivo grandmastera, ali je zahtevao trening na milionima partija i potpuno je neproziran: nemoguće je pročitati zašto je AI izabrao konkretan potez, što nije prihvatljivo za igru gde je konzistentnost i predvidivost ponašanja neprijatelja od suštinskog značaja.

Naše rešenje i prednosti

Ideja projekta je da se AI neprijatelja implementira korišćenjem **Drools rule engine-a**. Ovaj pristup donosi prednosti:

- **EksPLICITNA pravila** čitljiva kao poslovne politike. Ovo je posebno značajno u razvoju video igara zbog toga što omogućava dizajnerima da ručno unose pravila sa minimalnim znanjem programiranja. To olakšava ceo proces iterativnog razvoja jer se gubi potreba za prisustvom programera pri svakom testiranju novih parametara.
- **CEP (Complex Event Processing)** za detekciju obrazaca tokom borbenih događaja korišćenjem klizajućih prozora (sliding windows). Umesto da posmatramo konkretno vremensku odredbu, posmatraćemo prednanih N poteza i gledati da donesemo neki zaključak. Jedna od dobrih osobina neprijateljskog AI bila bi da se i adaptira na konkretno, specifično ponašanje igrača, na primer ukoliko je igrač u poslednjih nekoliko poteza samo koristio fizičke napade, možemo zaključiti da povećavanje odbrane ima veći značaj nego inače.
- **Backward chaining** za ciljno zaključivanje, poput toga "Kojim potezom/kombinacijom poteza bi mogao da ubije igrača". Na osnovu konkretnog cilja gleda kako može da ga ispuni.
- **Višeslojnu arhitekturu** (L1 percepcija → L2 taktika → L3 akcija) koja razdvaja odgovornosti i nudi fleksibilnost pri biranju konkretnog poteza

- **Arhetipovi** korišćenjem Template-a. Kako bi se bolje izrazile razlike u neprijateljima i njihove karakterne osobine (a bez dodatne komplikacije koja bi nasledila u ostalim pristupima), možemo koristiti template za podesavanje razlicitih pragova za činjenice. Na primer, za činjenicu `HEALTH_CRITICAL` , drugačiji prag bismo imali za goblina koji je plasiljiv (recimo 40%) i za viteza koji je neustrasiv (10%).

Metodologija rada

Ulazi u sistem (Input)

Za svaki potez neprijatelja sistem prima sledeće **ulazne činjenice** koje se upisuju u Drools radnu memoriju:

Činjenica	Polja	Sadržaj
EnemyFact	id, archetype, maxHp, hpPercent, currentStamina, maxStamina, currentMana, maxMana, attack, defense, magic	Kompletno stanje neprijatelja, uključujući arhitip i sve statistike
PlayerFact	currentHp, maxHp, attack, defense, magic	Kompletno stanje igrača
MoveOption	moveId, moveCategory, costType, costValue, projectedValue, dealsDamage	Jedan zapis po svakom potezu koji neprijatelj može priuštiti; <code>projectedValue</code> je unapred izračunata procenjena šteta; <code>dealsDamage</code> je <code>true</code> za prave damage efekte i za <code>steal</code> efekte koji ciljaju <code>resource: health</code> (jer <code>steal</code> po HP-u funkcionalno jeste šteta)
ActiveStatusEffect	target (entityId), statType, value	Aktivan status efekat na igraču ili neprijatelju (negativna vrednost = debuff)
CombatEventFact	type, source, target, moveCategory, amount, turnTimestamp	Događaj iz istorije borbe, unet u CEP entry-point " <code>combat-stream</code> " kao <code>@role(event)</code> . Filtrira se na ulazu — samo <code>MOVE_USED</code> i <code>DAMAGE_DEALT</code> se ubacuju u stream; ostali događaji (<code>RESOURCE_SPENT</code> , <code>STATUS_EFFECT_APPLIED</code> , <code>HEAL_RECEIVED</code> , ...) se preskaču. turnTimestamp je broj poteza : brojač se inkrementira na svaki

Činjenica	Polja	Sadržaj
		TURN_CHANGED događaj iz istorije, pa svi događaji u istom potezu dele isti timestamp. SessionPseudoClock napreduje 1ms po TURN_CHANGED -u, čime se CEP prozori (window:time(Nms) , this after[0ms, Nms]) izračunavaju u jedinicama poteza, ne u broju događaja. Pošto je borba uvek 1v1, source za DAMAGE_DEALT se izvodi iz target -a (suprotni karakter) jer engine sam ne piše actorId na te događaje

Izvedene činjenice koje sistem sam generiše primenom pravila:

Činjenica	Generiše je	Moguće vrednosti	Opis
CantFinishPlayer	ConfirmNoImmediateKill (L1)	marker (bez polja)	Insertuje se kada nijedr MoveOption ne može uk igrača ovaj potez; pokretač za sve L1 i CEP percepcijsk regule
ResourceStatus	L1 pravila	type= STARVED , resourceName= "stamina" ili "mana"	Resurs neprijatelja ispod 20% maksimuma
StatComparison	L1 pravila	PLAYER_PHYSICAL_DOMINATES , PLAYER_MAGIC_DOMINATES	Statistička prednost igrača nad neprijatelje
BurstDamageAssessment	CEP pravilo	severity= HIGH , total (ukupna šteta)	Igrač je nar > 35% neprijatelje maxHp u poslednjih događaja

Činjenica	Generiše je	Moguće vrednosti	Opis
PlayerBehaviorProfile	CEP pravila	PHYSICAL_SPAMMER , COMBO_PLAYER	Detektovan obrazac ponašanja igrača
PerceivedThreat	L1 pravila	level= CRITICAL (< 25% HP), HIGH (25–50% HP), LOW (≥ 50% HP)	Procenjeni nivo ugroženost neprijatelja osnovu trenutnog h a; uvek se generiše ta jedna vrednost
Tactic	L2 pravila (saliency 1–80)	MAXIMIZE_DAMAGE , SELF_BUFF , DEBUFF_PLAYER , CONSERVATIVE , DRAIN	Izabrana borbena taktika za tekući pote
EnemyDecision	L3 pravila / fallback	moveId, priority, reason	Izlaz sistema — odluka c potezu sa prioritetom tekstualnim obrazloženjem

Izlazi iz sistema (Output)

Sistem proizvodi jednu odluku po pozivu:

- **EnemyDecision** : identifikator poteza koji neprijatelj treba da odigra, sa prioritetom i tekstualnim obrazloženjem

Iz skupa svih generisanih odluka bira se ona sa najvišim prioritetom i vraća `CombatService` -u koji je izvršava.

Baza znanja

Popunjavanje baze znanja: Pravila su statički definisana u DRL fajlovima i učitavaju se pri pokretanju aplikacije. Konfiguracija likova, poteza i predmeta čita se iz JSON fajlova (`characters.json` , `moves.json` , `items.json`) — dodavanje novog neprijatelja ili poteza ne zahteva izmenu Java koda, već samo dodavanje u JSON fajl.

Kompletna lista pravila po DRL fajlovima:

level1-perception.dr1 — L1 Percepcija

Pravilo	Uslov	Akcija
ConfirmNoImmediateKill	Nijedna MoveOption(projectedValue ≥ playerHp) ne postoji	INSERT CantFinishPlayer()
EvaluateEnemyThreatCritical	CantFinishPlayer() + hpPercent < 0.25	INSERT PerceivedThreat(CRITIC
EvaluateEnemyThreatHigh	CantFinishPlayer() + 0.25 ≤ hpPercent < 0.50	INSERT PerceivedThreat(HIGH)
EvaluateEnemyThreatLow	CantFinishPlayer() + hpPercent ≥ 0.50	INSERT PerceivedThreat(LOW)
EvaluateResourceStaminaStarved	CantFinishPlayer() + stamina < 20% maksimuma	INSERT ResourceStatus(STARVED "stamina")
EvaluateResourceManaStarved	CantFinishPlayer() + mana < 20% maksimuma	INSERT ResourceStatus(STARVED
DetectStatDisadvantage	CantFinishPlayer() + player.attack – enemy.defense > 5	INSERT StatComparison(PLAYER_PHYSICA
DetectMagicThreat	CantFinishPlayer() + PlayerFact(magic ≥ 8)	INSERT StatComparison(PLAYER_MAGIC_D

accumulate-burst.dr1 — CEP akumulacija

Pravilo	Uslov	Akcija
AssessBurstDamageRisk	CantFinishPlayer() + Zbir player DAMAGE_DEALT u poslednjih ~3 poteza igrača (window:time(6ms) — pošto se igrač i neprijatelj smenjuju, 6 jedinica vremena pokriva otprilike 3 poteza igrača) > 35% neprijateljevog maxHp	INSERT BurstDamageAssessment(total, HIGH)

Zašto window:time umesto window:length ? Burst pravilo nas zanima u smislu *poslednjih N poteza*, bez obzira da li je igrač u nekom potezu uopšte naneo štetu. window:length(N) bi gledao samo

poslednjih N `DAMAGE_DEALT` događaja — što za potez bez napada ne bi „resetovalo" stari burst iz pre nekoliko poteza. `window:time(6ms)` u kombinaciji sa `turnTimestamp` koji je u jedinicama poteza daje pravi vremenski prozor.

cep-patterns.dr1 — CEP obrasci ponašanja

Pravilo	Uslov	Akcija
DetectPlayerPhysicalSpammer	CantFinishPlayer() + Igrač koristio <code>DAMAGE_PHYSICAL</code> $\geq$ 3 puta u poslednjih 4 odgovarajuća <code>MOVE_USED</code> događaja (<code>window:length(4)</code>)	INSERT PlayerBehaviorProfile(PHYSICAL_SPAMMER)
DetectPlayerComboSetup	CantFinishPlayer() + Igrač koristio <code>ENEMY_DEBUFF</code> pa zatim napad u narednih do 2 poteza (<code>this</code> <code>after[0ms, 2ms]</code>)	INSERT PlayerBehaviorProfile(COMBO_PLAYER)

Trojni mehanizam temporalnog rezonovanja u CEP-u:

Mehanizam	Šta meri	Gde se koristi
<code>over</code> <code>window:length(N)</code>	Poslednjih N događaja koji odgovaraju pattern-u (count-based, vreme ne učestvuje)	DetectPlayerPhysicalSpammer — interesuje nas broj fizičkih napada, ne vreme
<code>over</code> <code>window:time(Nms)</code>	Događaji koji odgovaraju pattern-u, čiji <code>@Timestamp</code> je unutar poslednjih N jedinica vremena u odnosu na sat sesije	AssessBurstDamageRisk — interesuje nas vremenski prozor (poslednjih ~3 poteza)
<code>this</code> <code>after[lower, upper]</code> <code>\$e1</code>	Uređena relacija između dva specifična događaja: drugi se desio između <code>lower</code> i <code>upper</code> jedinica vremena nakon prvog	DetectPlayerComboSetup — debuff <i>pa onda</i> napad (redosled bitan)

Window operacije agregiraju skup događaja (count/sum/avg), this after izražava sekvencu A-pa-B i ne može se zameniti prozorom.

backward-queries.dr1 — Queryji

Pomoćni queryji (direktan pattern match)

Ovi queryji su obični Drools predikati nad radnom memorijom — jednoslojna disjunkcija pattern-a, bez rekurzije i bez generisanja međucijeva. Drools ih razrešava jednim prolazom kroz činjenice, pa nisu *backward chaining* u užem smislu (graf ciljeva ne postoji), iako se sintaksno pozivaju isto, sa ? prefiksom.

Upit	Parametri	Dokazuje
canKillPlayer	playerHp	Postoji MoveOption čiji projectedValue >= playerHp
isVulnerableToPhysical	entityId	Entitet ima aktivan defense debuff (ActiveStatusEffect) Ili urođeno nisku odbranu (defense <= 4). Tri OR grane razlikuju izvor podatka: status efekat, PlayerFact ili EnemyFact . Query je samo za fizičku ranjivost — magijska otpornost u igri ne postoji, pa magijska ranjivost ne bi imala smisla.

Backward chaining — canKillPlayerInNTurns

Upit	Parametri
canKillPlayerInNTurns	playerHp, enemyMana, enemyStamina, turnsLeft

Postavlja cilj ("dokazati da neprijatelj može ubiti igrača za N poteza") i onda za svaki potez koji nanosi štetu, napravi "scenario" kakva bi situacija bila sa resursima i enemy helathom, i onda rekurzivno poziva sa tim novim stanjem sa N-1 poteza: ?canKillPlayerInNTurns(playerHp - \$dmg, regenClamp(enemyMana - manaCost(\$costType, \$cost) + \$mag, \$maxM), regenClamp(enemyStamina - staminaCost(\$costType, \$cost) + \$att, \$maxS), turnsLeft - 1;) .

Struktura query-ja:

```
query "canKillPlayerInNTurns"(int playerHp, int enemyMana, int enemyStamina, int turnsLeft)
// Bazni slučaj: igrač već poražen
eval(playerHp <= 0)
or
// Rekurzivni: postoji damaging potez M koji možemo priuštiti,
// i nakon njegove primene (+ regen za sledeći potez) cilj je dostižan u N-1 poteza
eval(turnsLeft > 0 && playerHp > 0)
$e : EnemyFact($mag : magic, $att : attack, $maxM : maxMana, $maxS : maxStamina)
$m : MoveOption($dmg : projectedValue, $cost : costValue, $costType : costType,
projectedValue > 0)
```



```

eval(canAfford($costType, $cost, enemyMana, enemyStamina))
// Pre-vezivanje međurezultata kroz `from Integer.valueOf(...)` da rekurzivni
// poziv prima isključivo proste declaration reference – videti odeljak
// "Otklanjanje grešaka kroz analizu logova" za detalje (zaobilazi
// QueryArgument indexing bug u Drools-u kada mešani izrazi idu kao argumenti).
Integer($newHp : intValue)    from Integer.valueOf(playerHp - $dmg)
Integer($newMana : intValue)  from Integer.valueOf(regenClamp(enemyMana    - manaCost($cos
Integer($newStam : intValue)  from Integer.valueOf(regenClamp(enemyStamina - staminaCost($
Integer($newTurns : intValue) from Integer.valueOf(turnsLeft - 1)
?canKillPlayerInNTurns($newHp, $newMana, $newStam, $newTurns; )
end

```

Kako Drools razrešava ovaj cilj:

- Bazni slučaj** — Drools prvo proverava prvu OR granu: ako je `playerHp <= 0` (igrač već mrtav), cilj je dokazan i query vraća uspeh.
- Rekurzivni slučaj** — ako bazni nije ispunjen, Drools razrešava drugu OR granu:
 - Pronalazi `EnemyFact` (jedinствен, daje konstante: `magic`, `attack`, `maxMana`, `maxStamina`)
 - Iterira kroz sve `MoveOption` činjenice koje su `damaging ( projectedValue > 0 )` i `affordable ( canAfford helper)`
 - Za svaki takav potez** Drools formira novi cilj sa transformisanim stanjem: `playerHp - dmg`, ažurirane resursi posle troška + regena za sledeći potez (`regenClamp` osigurava nenegativnost i ne prelazak `max-a`), `turnsLeft - 1`
 - Rekurzivno poziva `?canKillPlayerInNTurns` sa novim ciljem
 - Ako bilo koja grana stigne do baznog slučaja**, cela rekurzija uspeva i query vraća dokaz

Pomoćne funkcije (definisane u istom DRL fajlu):

Funkcija	Svrha
<code>canAfford(t, c, m, s)</code>	<code>true</code> ako enemy ima dovoljno resursa za potez tipa <code>t</code> cene <code>c</code>
<code>manaCost(t, c)</code>	Vraća <code>c</code> ako je <code>t == "mana"</code> , inače <code>0</code> (selektivno trošenje)
<code>staminaCost(t, c)</code>	Vraća <code>c</code> ako je <code>t == "stamina"</code> , inače <code>0</code>
<code>regenClamp(v, max)</code>	<code>Math.max(0, Math.min(max, v))</code> — klampuje resurs nakon regena

Šta čini ovo pravim backward chaining-om:

- Cilj se ne dokazuje direktno**, već se dekomponuje na potcijeve (svaki potez = novi međucilj)
- Generiše se proof tree**: koren je početni cilj, grananje je po svakom potezu, listovi su bazni slučajevi
- Pretraga je depth-first sa unifikacijom parametara** — Drools razrešava jednu po jednu granu, vraća se nazad ako grana ne uspe

- **Drools sam upravlja rekurzijom** — nema iterativne kontrole iz Jave; samo definisanje queryja u DRL-u opisuje *šta* je dokaz, ne *kako* tražiti

Konkretni primer rekurzivnog razvijanja:

Za GoblinMage sa mana=45, magic=9; player HP=60; potezi `firebolt` (15 mane, 20 dmg) i `arcaneSurge` (20 mane, 25 dmg) — poziv `?canKillPlayerInNTurns(60, 45, 20, 3)`:

```
?canKill(60, 45, 20, 3)
├ Grana firebolt: ?canKill(40, 39, 24, 2)           // 45-15+9=39 mane za sledeći potez
│ └ Grana firebolt: ?canKill(20, 33, 28, 1)
│   └ Grana firebolt: ?canKill(0, 27, 32, 0) → eval(playerHp<=0) ✓ DOKAZ
│     └ ...
│   └ ...
└ Grana arcaneSurge: ?canKill(35, 34, 24, 2)
  └ ...
```

Pošto je dovoljno da jedna grana stigne do baznog slučaja, čim Drools nađe putanju `firebolt → firebolt → firebolt` koja zadovoljava, query vraća uspeh i ostatak stabla se ne istražuje.

Upotreba u sistemu — koristi se na dva mesta u `level2-tactics.drl`:

- `ChooseFinishSoonTactic` (salience 60): pri `PerceivedThreat(LOW)` poziva `?canKillPlayerInNTurns($hp, $em, $es, 3; )` — ako postoji 3-turn kill plan, prelazi u `MAXIMIZE_DAMAGE`
- `ChooseConservativeTactic` (salience 45): poziva `not ?canKillPlayerInNTurns(...; )` kao guard — blokira ulaz u konzervativni mod ako postoji put do pobede

level2-tactics.drl — L2 Selekcija taktike

Pravilo	Salience	Uslov	
<code>ChooseDrainOnHighBurst</code>	80	<code>BurstDamageAssessment(HIGH) + DRAIN</code> potez dostupan	INS
<code>ChooseDebuffOnHighBurst</code>	79	<code>BurstDamageAssessment(HIGH) + ENEMY_DEBUFF</code> potez dostupan	INS Tac
<code>ChooseSelfBuffOnHighBurst</code>	78	<code>BurstDamageAssessment(HIGH) + SELF_BUFF</code> potez dostupan	INS Tac
<code>ChooseConservativeOnHighBurst</code>	77	<code>BurstDamageAssessment(HIGH)</code> , nijedna druga opcija nije dostupna	INS Tac
<code>ChooseFinishSoonTactic</code>	60	<code>PerceivedThreat(LOW) + ?canKillPlayerInNTurns(playerHp, enemyMana, enemyStamina, 3)</code> uspešan	INS Tac

Pravilo	Salience	Uslov	
ChooseDebuffPlayerTactic	50	StatComparison(PAYER_PHYSICAL_DOMINATES ili PAYER_MAGIC_DOMINATES) + ENEMY_DEBUFF dostupan	INS Tac
ChooseConservativeTactic	45	ResourceStatus(STARVED) + not ? canKillPlayerInNTurns(playerHp, enemyMana, enemyStamina, 3) (blokirano kad postoji kill plan)	INS Tac
ChooseDrainTactic	40	PerceivedThreat(CRITICAL ili HIGH) + DRAIN dostupan	INS
ChooseSelfBuffTactic	35	PerceivedThreat(LOW ili HIGH) + SELF_BUFF dostupan	INS Tac

level3-action.dr1 — L3 Selekcija poteza

Pravilo	Uslov	Akcija
EmitImmediateKillMove	?canKillPlayer(playerHp) uspešan (direktno iz ulaznih činjenica; uzajamno isključivo sa ConfirmNoImmediateKill)	INSERT EnemyDecision(le potez, prioritet
EmitMaximizeDamagePhysicalVulnerable	Tactic(MAXIMIZE_DAMAGE) + ? isVulnerableToPhysical("player") + not EnemyDecision(priority >= 10)	INSERT EnemyDecision(na fizički potez, prioritet 10)
EmitMaximizeDamageAny	Tactic(MAXIMIZE_DAMAGE) + not EnemyDecision(priority >= 9) + MoveOption(dealsDamage == true)	INSERT EnemyDecision(na potez koji nanosi štetu, prioritet

Zašto se EmitMaximizeDamageAny poziva na dealsDamage umesto na kategoriju poteza? U preglednom modelu, DRAIN je posebna kategorija od DAMAGE_*, ali kada steal efekat cilja resource: health (npr. drainLife kod veštice), on **istovremeno** smanjuje HP protivniku i leči izvođača — funkcionalno identično napadu. Pravilo koje bira „najbolji potez koji nanosi štetu" mora da računa i takve poteze, inače bi MAGIC_DRAINER arhetip (čiji su svi napadi DRAIN tipa) propao u fallback iako ima savršeno valjan damage potez. dealsDamage polje je izvedeno u

DroolsEnemyAIService.dealsDamage() na osnovu strukture poteza — true za prave damage efekte i za steal po HP-u. Ne diramo MoveCategory jer i dalje treba da znamo da je drainLife formalno DRAIN za potrebe ChooseDrainTactic i EmitDrainMove .

Zašto nema `EmitMaximizeDamageMagicVulnerable` ? U igri ne postoji magijska otpornost — magijska šteta se ne reducira nikakvim statom protivnika, dok se fizička reducira odbranom (armorom). Zato „fizička ranjivost“ (`isVulnerableToPhysical`) ima konkretno značenje (nizak `defense` ili aktivan `defense-debuff`), dok „magijska ranjivost“ ne bi imala uporište u mehanici igre — magijski potez uvek nanosi istu štetu bez obzira na metu, pa `EmitMaximizeDamageAny` pokriva taj slučaj jednako dobro. |

```
EmitSelfBuffMove | Tactic(SELF_BUFF) + not EnemyDecision(priority >= 7) | INSERT
EnemyDecision(self-buff potez, prioritet 7) || EmitDebuffMove | Tactic(DEBUFF_PLAYER) + not
EnemyDecision(priority >= 7) | INSERT EnemyDecision(debuff potez, prioritet 7) ||
EmitDrainMove | Tactic(DRAIN) + not EnemyDecision(priority >= 7) | INSERT EnemyDecision(drain
potez, prioritet 7) || EmitConservativeMove | Tactic(CONSERVATIVE) + not
EnemyDecision(priority >= 6) | INSERT EnemyDecision(najjeftiniji potez, prioritet 6) |
```

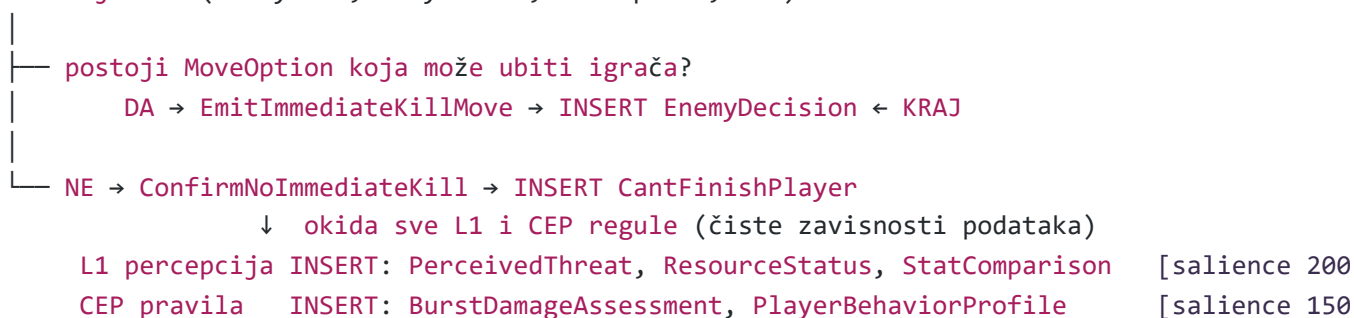
`fallback.dr1` — Fallback pravila

Pravilo	Salience	Uslov	Akcija
<code>ChooseFallbackTactic</code>	-10 (L2)	<code>CantFinishPlayer()</code> + nijedna taktika nije izabrana	INSERT <code>Tactic(MAXIMIZE_DAMAGE)</code>
<code>EmitFallbackDecision</code>	-10 (L3)	<code>Tactic()</code> postoji + nijedna odluka nije generisana	INSERT <code>EnemyDecision(najjeftiniji dostupni potez, prioritet 1)</code>

Zašto negativni salience na fallback-u? L1 perceptivna pravila i CEP pravila imaju podrazumevani salience 0; L2 taktička pravila imaju salience 35–80. Da je `ChooseFallbackTactic` imao pozitivan salience (recimo 1), upalio bi se pre L1 percepcije — pošto `CantFinishPlayer()` aktivira agendu, sva ta pravila se nadmeću, a najveći salience pobeđuje. Time bi se `Tactic(MAXIMIZE_DAMAGE)` ubacila pre nego što su `BurstDamageAssessment`, `PerceivedThreat`, `StatComparison` uopšte postojali, i `not Tactic()` guard bi blokirao sve L2 taktike koje zavise od tih percepcijskih činjenica. Negativan salience (-10) osigurava da fallback ide **poslednji** — samo ako stvarno nijedno drugo pravilo nije proizvelo taktiku/odluku.

Mehanizam forward chaining-a — Java kod samo upisuje ulazne činjenice i poziva `session.fireAllRules()`. Ne postoji nikakvo eksplicitno određivanje redosleda faza. Kaskada između slojeva nastaje isključivo kroz zavisnosti podataka, sa jasnom rasviljicom na samom početku:

Ulazne činjenice (`EnemyFact`, `PlayerFact`, `MoveOption`, ...)



↓ nove činjenice okidaju L2 pravila

L2 taktika INSERT: *Tactic* [salience 35–

↓ *Tactic* okida L3 pravila

L3 akcija INSERT: *EnemyDecision* (prioritet 6–10) [salience 0]

↓ ako ništa nije proizvelo *Tactic/EnemyDecision*

Fallback INSERT: *Tactic*(MAXIMIZE_DAMAGE) / *EnemyDecision*(prio 1) [salience –10
← KRAJ

Svaki sloj je aktiviran činjenicama koje je prethodni sloj upisao.

Saliencee tieri (od najvišeg ka najnižem prioritetu u agendi):

Tier	Pravila	Salience	Svrha
Template critical	EvaluateHealthCritical_<ARCH>	100	Arhetipsko HP-critical pravilo prečiće univerzalni EvaluateEnemyThreat*
L2 burst-reaction	ChooseDrainOnHighBurst , ChooseDebuffOnHighBurst , ChooseSelfBuffOnHighBurst , ChooseConservativeOnHighBurst	80 / 79 / 78 / 77	Kada BurstDamageAssessment(HIGH) postoji, biraju taktiku po prioritetu sa silaznim padom
L2 ostale taktike	ChooseFinishSoonTactic , ChooseDebuffPlayerTactic , ChooseConservativeTactic , ChooseDrainTactic , ChooseSelfBuffTactic	60 / 50 / 45 / 40 / 35	Selekcija taktike na osnovu percepcijskih činjenica
Template aggressive	ChooseAggressiveTactic_<ARCH>	50	Arhetip uvodi MAXIMIZE_DAMAGE kada je na komfornom HP-u
L1 percepcija	sva pravila iz level1-perception.dr1	200	Sva percepcija mora da završi pre L2 selekcije taktike — videti odeljak "Otklanjanje grešaka kroz analizu logova"
CEP	sva pravila iz cep-patterns.dr1 i accumulate-burst.dr1	150	Akumulativne i temporalne činjenice (burst, behavior profili) pre L2 selekcije
L3 akcija	sva pravila iz level3-action.dr1	0 (default)	Ne nadmeću se međusobno — priority polje u EnemyDecision + not EnemyDecision(priority >= N) guard

Tier	Pravila	Salience	Svrha
Fallback	ChooseFallbackTactic , EmitFallbackDecision	-10	Idu poslednji — samo ako stvarno nijedan drugi mehanizam nije proizveo izlaz

- `EmitImmediateKillMove` i `ConfirmNoImmediateKill` su **uzajamno isključivi** čistim zavisnostima podataka; kada ubistvo postoji, uslov `not MoveOption(projectedValue >= playerHp)` u `ConfirmNoImmediateKill` nikad nije ispunjen; kada ne postoji, `query ?canKillPlayer` u `EmitImmediateKillMove` ne nalazi dokaz
- L1 i CEP pravila pišu **različite činjenice** i ne takmiče se međusobno; redosled njihovog paljenja ne utiče na ishod
- L2 burst pravila (salience 77–80) koriste salience za prioritet: `ChooseDrainOnHighBurst` je poželjniji od `ChooseSelfBuffOnHighBurst`; `not Tactic()` guard deaktivira ostala L2 pravila čim jedno uspe
- L3 pravila — selekcija je preko `priority` polja u `EnemyDecision` faktu; `not EnemyDecision(priority >= N)` guard sprečava prepisivanje odluke višeg prioriteta; `bestDecision()` na kraju bira onu sa najvišim prioritetom

Template-i i arhetipovi

Različiti tipovi neprijatelja moraju da reaguju različito na istu situaciju: vitez od 30% HP ne bi trebalo da paniči kao goblin od 30% HP. Umesto da pišemo zasebna pravila za svaki arhetip (što duplira logiku) ili da jednim globalnim pragom `HEALTH_CRITICAL` ukinemo karakterizaciju, koristimo **programatski generisana pravila po arhetipu**: jedna struktura pravila se parametrizuje različitim vrednostima pragova. Implementacija je u `DroolsConfig.generateArchetypeDrl()` — Java metoda koja pri pokretanju aplikacije konstruiše `template-archetypes.drl` na osnovu niza `ARCHETYPES`, dodaje ga u Drools `KieFileSystem` zajedno sa statičkim DRL-ovima, i build prolazi normalno kroz `KieBuilder`.

Podržani arhetipovi i njihovi pragovi (definisano u `DroolsConfig.ARCHETYPES`):

Arhetip	<code>criticalHpPct</code>	<code>retreatHpPct</code>	<code>aggressionHpPct</code>	Karakter
PHYSICAL_BRAWLER	0.33	0.25	0.50	Tipičan ratnik — paniči srednje rano
MAGIC_CASTER	0.30	0.30	0.45	Krhki čarobnjak — ne juri agresivno
MAGIC_DRAINER	0.35	0.35	0.50	Drain specijalista —

Arhetip	criticalHpPct	retreatHpPct	aggressionHpPct	Karakter
				oprezno bira borbe
PHYSICAL_SKIRMISHER	0.30	0.20	0.55	Brz i preduzimljiv — agresivan dok god ima HP
BALANCED_TANK	0.25	0.15	0.35	Izdržljiv — kasno paniči, kreće u napad i kad je povređen

Mapiranje konkretnih neprijatelja na arhetipove drži `DroolsEnemyAIService.ARCHETYPE_MAP` : na primer, `goblinWarrior / minotaur / centaur` SU `PHYSICAL_BRAWLER` , `witch / archdemon` SU `MAGIC_DRAINER` , `dragon / skeletonArcher` SU `BALANCED_TANK` . Više različitih neprijatelja deli isti arhetip i time istu logiku donošenja odluka, ali se vizuelno i mehanički razlikuju u ostatku igre.

Pravila koja se generišu po arhetipu: za svaki red u tabeli iznad `generateArchetypeDr1()` proizvodi dva pravila sa identičnom strukturom, samo se konstante razlikuju:

Generisano pravilo	Salience	Uslov	Akci
<code>EvaluateHealthCritical_<ARCHETYPE></code>	100 (L1)	<code>EnemyFact(archetype == <ARCHETYPE>, hpPercent < criticalHpPct) + not PerceivedThreat()</code>	INSERT <code>PerceivedThreat</code>
<code>ChooseAggressiveTactic_<ARCHETYPE></code>	50 (L2)	<code>EnemyFact(archetype == <ARCHETYPE>, hpPercent >= aggressionHpPct) + not PerceivedThreat(CRITICAL) + not Tactic()</code>	INSERT <code>Tactic(MAXIMIz</code>

Konkretno, za `MAGIC_CASTER` arhetip generiše se (sa `criticalHpPct = 0.30` i `aggressionHpPct = 0.45`):

```
rule "EvaluateHealthCritical_MAGIC_CASTER"
  salience 100
  when
    EnemyFact(archetype == Archetype.MAGIC_CASTER, hpPercent < 0.30)
    not PerceivedThreat()
```



```

then
    insert(new PerceivedThreat(ThreatLevel.CRITICAL));
end

rule "ChooseAggressiveTactic_MAGIC_CASTER"
    salience 50
when
    EnemyFact(archetype == Archetype.MAGIC_CASTER, hpPercent >= 0.45)
    not PerceivedThreat(level == ThreatLevel.CRITICAL)
    not Tactic()
then
    insert(new Tactic(TacticType.MAXIMIZE_DAMAGE));
end

```

Ista struktura, samo različite konstante (`Archetype.<X>` , `criticalHpPct` , `aggressionHpPct`), ponavlja se za svih 5 arhetipova — što na izlazu daje 10 dodatnih pravila u baze znanja.

Interakcija sa univerzalnim L1 pravilima: statički `EvaluateEnemyThreatCritical/High/Low` iz `level1-perception.drl` koriste fiksne pragove (0.25 / 0.50) i okidani su `CantFinishPlayer()`. Template-ovana `EvaluateHealthCritical_<X>` pravila imaju **viši salience (100)** i koriste arhetip-specifičan prag — kada se obojica mogu primeniti, template-ovano pravilo pali prvo i `not PerceivedThreat()` guard sprečava univerzalno pravilo da prepíše rezultat. Tako arhetip dobija prioritet, ali ako njegov prag nije dosegnut, sistem se vraća na univerzalnu logiku kao bezbedan default.

Prednost u praksi: dodavanje novog arhetipa (npr. `BERSERKER` koji nikad ne paniči i agresivno napada: `criticalHpPct = 0.10` , `aggressionHpPct = 0.20`) zahteva:

1. dodavanje enum vrednosti `BERSERKER` u `Archetype.java`
2. dodavanje jednog reda u `DroolsConfig.ARCHETYPES`
3. (opcionally) mapiranje konkretnog neprijatelja u `ARCHETYPE_MAP`

Bez pisanja novih DRL pravila, bez izmene postojeće logike, bez ponavljanja koda. Template-i ovde služe i kao **mehanizam za izražavanje karakternosti neprijatelja** — ista borbena situacija proizvodi različite percepcije i različite taktike u zavisnosti od toga ko je u njoj, što je suštinski element kvalitetne AI u strateškim igrama.

Šta se template-uje, a šta ne: nije sve parametrizovano po arhetipu. Pragove vezane za HP (kritičnost, agresivnost) ima smisla razlikovati — to je deo karaktera neprijatelja. Ali generička percepcija kao što je „mana je pala ispod 20%" ili „igrač fizički dominira" je univerzalna i živi u statičkim L1 pravilima (`level1-perception.drl`). Template-i pokrivaju samo onu dimenziju gde arhetip donosi razliku.

Konkretan primer rezonovanja

Scenario

Neprijatelj: GoblinMage, nivo 2 (arhitip: MAGIC_CASTER)

- HP: 30 / 70 (43%) | Mana: 42 / 45 | Stamina: 3 / 20
- Statistike: napad=4, odbrana=3, magija=9
- Dostupni potezi:
 - firebolt : DAMAGE_MAGIC, cena: 15 mane
 - arcaneSurge : DAMAGE_MAGIC, cena: 20 mane
 - hexShield : SELF_BUFF, cena: 10 mane
 - manaDrain : DRAIN, cena: 10 stamine → **nije dostupan** (stamina=3 < 10)
- Nema aktivnih statusnih efekata

Igrač: Knight, nivo 2

- HP: 60 / 120 (50%) | Statistike: napad=12, odbrana=10, magija=4
- Nema aktivnih statusnih efekata

Istorija borbe (poslednja 4 događaja na `combat-stream`):

Redosled	Akter	Potez	Kategorija	Šteta
1	Igrač	slash	DAMAGE_PHYSICAL	12
2	Igrač	slash	DAMAGE_PHYSICAL	14
3	Igrač	battleCry	SELF_BUFF	—
4	Igrač	slash	DAMAGE_PHYSICAL	11

Korak 1 — Provera ubojstva: `ConfirmNoImmediateKill`

Pravila `ConfirmNoImmediateKill` i `EmitImmediateKillMove` pale direktno iz ulaznih činjenica i uzajamno su isključiva, samo jedno od njih može da pali u datom potezu.

`ConfirmNoImmediateKill` proverava: postoji li `MoveOption` čiji `projectedValue` $\geq$ `player.currentHp` ?

Potez	Projektivna šteta	Player currentHp	Može ubiti?
firebolt	20	60	Ne, $20 < 60$
arcaneSurge	25	60	Ne, $25 < 60$
hexShield	— (nije šteta)	60	Ne

Nijedna `MoveOption` ne zadovoljava uslov → `ConfirmNoImmediateKill` pali → **INSERT**
`CantFinishPlayer()` ← okidač za sve L1 i CEP regule

Da je igrač imao, recimo, 22 HP, `arcaneSurge` bi zadovoljio uslov: `ConfirmNoImmediateKill` ne bi palilo, već `EmitImmediateKillMove` — i odmah bi bila emitovana odluka sa prioritetom 100, bez prolaska kroz L2 i L3.

Korak 2 — L1 Percepcija: okidač `CantFinishPlayer`

Upisom `CantFinishPlayer()` u koraku 1, sva L1 pravila postaju aktivna.

EvaluateEnemyThreatHigh :

$hpPercent = 30/70 = 0.43 \rightarrow 0.25 \leq 0.43 < 0.50 \checkmark \rightarrow \text{INSERT } PerceivedThreat(HIGH) \leftarrow$ okidač za drain/self-buff taktike

EvaluateResourceStaminaStarved :

$stamina\ 3/20 = 15\% < 20\% \rightarrow \text{INSERT } ResourceStatus(STARVED, "stamina")$

EvaluateResourceManaStarved :

$mana\ 42/45 = 93\% > 20\% \rightarrow$ uslov nije ispunjen

DetectStatDisadvantage :

$player.attack(12) - enemy.defense(3) = 9 > 5 \rightarrow \text{INSERT } StatComparison(PAYER_PHYSICAL_DOMINATES)$

Korak 3 — CEP nad event stream-om: okidač `CantFinishPlayer`

CEP pravila takođe zahtevaju `CantFinishPlayer()` i procesiraju tok borbenih događaja korišćenjem klizajućih prozora.

AssessBurstDamageRisk (accumulate sum, window: poslednjih ~3 poteza igrača — `window:time(6ms)`):

sum DAMAGE_DEALT od igrača u vremenskom prozoru: $12 + 14 + 11 = 37$ $37 > 70 \times 0.35 = 24.5$
 $\checkmark \rightarrow \text{INSERT } BurstDamageAssessment(37, HIGH)$

DetectPlayerPhysicalSpammer (accumulate count, window: poslednja 4 poteza):

DAMAGE_PHYSICAL potezi igrača: slash, slash, battleCry (ne), slash = $3 \geq 3 \checkmark \rightarrow \text{INSERT } PlayerBehaviorProfile(PHYSICAL_SPAMMER)$

Korak 4 — L2 Selekcija taktike: aktivirana `BurstDamageAssessment` činjenicom

Upisom `BurstDamageAssessment(37, HIGH)` u koraku 2, burst pravila postaju aktivna. Sa salienceom 77–80 — višim od ostalih L2 pravila — evaluiraju se prva.

ChooseDrainOnHighBurst (salience 80):

`BurstDamageAssessment(HIGH) ✓ | MoveOption(DRAIN) dostupan? → manaDrain nije dostupan (stamina=3 < 10) → uslov nije ispunjen`

ChooseDebuffOnHighBurst (salience 79):

`BurstDamageAssessment(HIGH) ✓ | MoveOption(ENEMY_DEBUFF) dostupan? → GoblinMage nema debuff poteze → uslov nije ispunjen`

ChooseSelfBuffOnHighBurst (salience 78):

`BurstDamageAssessment(HIGH) ✓ | MoveOption(SELF_BUFF) dostupan? → hexShield je SELF_BUFF ✓ | not Tactic() ✓ → INSERT Tactic(SELF_BUFF)`

Preostala L2 pravila:

- `ChooseDebuffPlayerTactic` (50): `StatComparison` ✓ ali nema `ENEMY_DEBUFF` → ne pali
- `ChooseConservativeTactic` (45): `ResourceStatus(STARVED)` ✓, ali `not Tactic()` **ne važi** → ne pali

Korak 5 — L3 Selekcija poteza: aktivirana `Tactic` činjenicom

Upisom `Tactic(SELF_BUFF)`, pravilo `EmitSelfBuffMove` postaje aktivno.

EmitSelfBuffMove :

`Tactic(SELF_BUFF) ✓ | not EnemyDecision(priority >= 7) ✓ MoveOption(SELF_BUFF) = hexShield → INSERT EnemyDecision("hexShield", 7, "Self-buff tactic: hexShield")`

Ishod i obrazloženje

GoblinMage odabira `hexShield`, buff-uje se umesto da napada.

Odluka je nastala kroz čisti forward chaining — svaki korak je bio automatski okidan novim činjenicama:

1. `PlayerFact + MoveOption` → `ConfirmNoImmediateKill` pali → kill nije moguć → **INSERT** `CantFinishPlayer`
2. `CantFinishPlayer` → L1 pali → **INSERT** `PerceivedThreat(HIGH)`, `ResourceStatus(STARVED)`, `StatComparison`
3. `CantFinishPlayer` → CEP pali → **INSERT** `BurstDamageAssessment(HIGH)`, `PlayerBehaviorProfile(PHYSICAL_SPAMMER)`

4. `BurstDamageAssessment(HIGH) → ChooseSelfBuffOnHighBurst` pali → `INSERT` `Tactic(SELF_BUFF)`

5. `Tactic(SELF_BUFF) → EmitSelfBuffMove` pali → `INSERT` `EnemyDecision("hexShield", 7, ...)`

Primećujemo da `ChooseConservativeTactic` (salience 45) **nije** palilo, iako je `ResourceStatus(STARVED)` bio prisutan — burst pravilo (salience 78) je ubacilo `Tactic` pre nego što je konzervativno pravilo imalo prilike. `not Tactic()` guard je potom automatski deaktivirao sva preostala L2 pravila.

Ceo proces, od sirovih statistika do konkretnog poteza, odvija se transparentno kroz čitljiva pravila, sa jasnim tragom zaključivanja koji je moguće ispitati i promeniti u svakom koraku.

Otklanjanje grešaka kroz analizu logova

Tokom integracionog testiranja sistema kroz nekoliko realnih borbi, `RuleFiringLogger` (zakačen na svaki `KieSession`) beleži svaki `INSERT` i `FIRED` događaj sa milisekundnom vremenskom oznakom. Pažljivo čitanje tih logova otkrilo je dva strukturna problema u prvobitnoj implementaciji i jedan rubni slučaj u Java fallback-u. Sam proces dijagnostike — poređenje stvarnog redosleda paljenja sa onim koji arhitektura nalaže, identifikovanje mesta gde se sistem ponaša drugačije — odlična je ilustracija transparentnosti rule-based pristupa: svaka odluka je trag pravila i činjenica koji se čita unazad, bez black-box komponente ili neprozirne heuristike.

Bug 1 — `ArrayIndexOutOfBoundsException` u rekurzivnom backward chaining-u

Simptom (log za `giantSpider`, `PerceivedThreat(LOW)`, `ChooseFinishSoonTactic` pokušava da se aktivira):

```
17:48:48.184 FIRED EvaluateEnemyThreatLow ← PerceivedThreat(LOW) ubačen
17:48:48.205 ERROR ArrayIndexOutOfBoundsException: Index 6 out of bounds for length 4
           at org.drools.base.rule.QueryArgument.evaluateDeclaration(QueryArgument.java
           at org.drools.core.reteoo.QueryElementNode.createDroolsQuery(QueryElementNod
```

Crash se javljao samo na LOW-threat putanji jer je to jedina putanja koja okida `?`

`canKillPlayerInNTurns(...)`. Sesije sa HIGH ili CRITICAL pretnjom su prolazile bez problema (nikad nisu pokretale ovaj query).

Uzrok: prvobitni rekurzivni poziv u `canKillPlayerInNTurns` mešao je parametre queryja (`playerHp`, `enemyMana`, `enemyStamina`, `turnsLeft`) sa lokalno vezanim deklaracijama (`$dmg`, `$cost`, `$costType`, `$mag`, `$att`, `$maxM`, `$maxS`) unutar aritmetičkih i function-call izraza:

```
?canKillPlayerInNTurns(
    playerHp - $dmg,
    regenClamp(enemyMana - manaCost($costType, $cost) + $mag, $maxM),
```

```
regenClamp(enemyStamina - staminaCost($costType, $cost) + $att, $maxS),
turnsLeft - 1; )
```

Drools `QueryArgument` kompajlira svako mesto argumenta kao literal-ili-declaration referencu, i pri kombinaciji više deklaracija iz različitih izvora unutar složenih izraza, internal indexing tuple-a izlazi iz opsega queryja (4 parametra) — index 6 odgovara sedmoj lokalnoj deklaraciji (`MoveOption` veze) koja u tuple-u nije bila.

Rešenje: pre-vezivanje međurezultata kroz `Integer.valueOf(...)` `from`, tako da rekurzivni poziv prima isključivo proste declaration reference (vidi izmenjeni query u odeljku `backward-queries.dr1` iznad):

```
Integer($newHp : intValue)    from Integer.valueOf(playerHp - $dmg)
Integer($newMana : intValue)  from Integer.valueOf(regenClamp(enemyMana    - manaCost($costTyp
Integer($newStam : intValue)  from Integer.valueOf(regenClamp(enemyStamina - staminaCost($cost
Integer($newTurns : intValue) from Integer.valueOf(turnsLeft - 1)
?canKillPlayerInNTurns($newHp, $newMana, $newStam, $newTurns; )
```

Crash je eliminisan; query se sada konzistentno evaluira bez izuzetka, a logika rekurzije (depth-first sa unifikacijom — vidi šemu iznad) ostala je netaknuta.

Bug 2 — Redosled paljenja: L2 taktike preteknu L1 percepciju

Simptom (log za goblinMage, igrač upravo naneo 23 štete = 57.5% maxHp neprijatelja):

06.320	FIRE	ConfirmNoImmediateKill	← ubaci CantFinishPlayer
06.326	FIRE	EvaluateEnemyThreatHigh	← ubaci PerceivedThreat(HIGH)
06.355	FIRE	ChooseSelfBuffTactic	← L2 odmah pali – Tactic = SELF_BUFF
06.367	FIRE	EmitSelfBuffMove	← L3 emituje arcaneSurge, odluka zaključana
06.393	FIRE	DetectMagicThreat	← L1 percepcija prekasno (StatComparison neisk)
06.422	FIRE	AssessBurstDamageRisk	← BurstDamageAssessment(HIGH) prekasno

Player ima `magic=13` (≥ 8 , trebalo da okine `PLAYER_MAGIC_DOMINATES`) i upravo je izvršio burst preko 35% praga (trebalo da okine `BurstDamageAssessment(HIGH)`). Obe činjenice je trebalo da oblikuju izbor taktike kroz burst-reactive pravila (salience 77–80), ali nijedna nije stigla na vreme —

`Tactic(SELF_BUFF)` je već bio upisan kroz nisko-saliencnu `ChooseSelfBuffTactic` (35), i `not Tactic()` guard u svim L2/L3 pravilima je blokirao kasnije evaluacije.

Uzrok: L1 perceptivna i CEP pravila imala su **podrazumevani salience 0**, dok L2 taktička pravila imaju salience 35–80. Drools agenda fire-uje po salience-u — čim jedno L1 pravilo upisuje `PerceivedThreat`, L2 pravilo koje ga odgovara (sa salience-om ≥ 35) preteže ostala L1/CEP pravila (salience 0). Faza $L1 \rightarrow L2 \rightarrow L3$, koju je arhitektura nalagala kao striktnu kaskadu, u praksi se preplitala.

Rešenje: eksplicitan salience za perceptivni i CEP sloj, znatno iznad bilo koje taktike:

Sloj	Stari salience	Novi salience	Zašto
L1 percepcija	0 (default)	200	Sva percepcija mora da završi pre L2
CEP (burst + behavior)	0 (default)	150	Akumulativne/temporalne činjenice pre L2
L2 burst-reactive	77–80	(nepromenjen)	Sada uspešno pretežu standardne taktike
L2 standardne	35–60	(nepromenjeno)	
L3 + fallback	0 / –10	(nepromenjeno)	Ne nadmeću se po salience-u, već po priority polju

Nakon promene, nova logovska sekvenca tačno odražava arhitektonsku nameru. Za istu situaciju (goblinWarrior u kasnijem testu) sada vidimo:

```
ConfirmNoImmediateKill → CantFinishPlayer
EvaluateEnemyThreatLow → PerceivedThreat(LOW)
DetectStatDisadvantage → StatComparison(PLAYER_PHYSICAL_DOMINATES)
AssessBurstDamageRisk → BurstDamageAssessment(HIGH, total=26)
ChooseSelfBuffOnHighBurst (sal 78) → Tactic(SELF_BUFF) ← burst-reactive sada pobeđuje
EmitSelfBuffMove → EnemyDecision(frenzy)
```

Svih 5 perceptivnih/CEP pravila završi pre nego što ijedno L2 dobije priliku. Burst-reactive pravila (77–80) sada uspešno pretežu standardne taktike (35–60) kada `BurstDamageAssessment(HIGH)` postoji.

Pass kao izlaz: neprijatelj koji nema nijedan dostupan potez

Treća situacija primećena u logovima: kada centaur u kasnoj fazi borbe ima stamina=3, a sva njegova fizička napada koštaju ≥ 10 , `insertFacts` u `DroolsEnemyAIService` (filtrira move-ove kroz `canAfford`) odbacuje sve poteze i **nijedna `MoveOption` činjenica se ne ubacuje** u radnu memoriju. Pipeline ipak ide do kraja jer L1 i CEP pravila ne zavise od `MoveOption`:

```
ConfirmNoImmediateKill → CantFinishPlayer
EvaluateEnemyThreat* → PerceivedThreat
AssessBurstDamageRisk → BurstDamageAssessment(HIGH)
ChooseConservativeOnHighBurst → Tactic(CONSERVATIVE)
— pickMove end: selected=maulSwing ← Java fallback bira NEDOSTUPAN potez!
```

Sva L3 emit pravila (uključujući salience –10 `EmitFallbackDecision`) zahtevaju **bar jednu `MoveOption`** za pattern match. Nijedna ne pali. Pre ispravke, Java fallback u `DroolsEnemyAIService.bestDecision()`

vraćao je prvi potez iz `enemy.getMoves()` bez obzira na dostupnost — što je rezultiralo time da se sistem trudio da izvrši potez koji ne može.

Rešenje: `bestDecision()` sada vraća literal `"pass"` kada se nije generisao nijedan `EnemyDecision`. Uslov `decisions.isEmpty()` je tačno onda kada nije ubačena nijedna `MoveOption` (jer i ultimate fallback `EmitFallbackDecision` zahteva pattern match), što direktno znači da neprijatelj nema dostupan potez. Postojeći `try { combatService.executeMove(...) } catch (InvalidMoveException)` blok u `CombatSessionService.processEnemyTurn` već konvertuje takav slučaj u clean `MOVE_USED` događaj sa `moveId: "pass"` — neprijatelj korektno preskoči potez, regen se primeni, i igrač nastavlja normalno.

```
if (!decisions.isEmpty()) {
    return decisions.stream()
        .max(Comparator.comparingInt(EnemyDecision::getPriority))
        .map(EnemyDecision::getMoveId)
        .orElse("pass");
}
// Nema EnemyDecision ⇔ nema affordable MoveOption ⇔ pass
return "pass";
```

Šta ovo govori o pristupu

Sva tri problema su otkrivena čitanjem logova i analizom redosleda paljenja pravila — nijedna od izmena nije zahtevala izmenu test-suite-a ili re-trening modela. Sam dijagnostički proces (čitam log, vidim da `DetectMagicThreat` pali **posle** `EmitSelfBuffMove` umesto pre, izračunam očekivani vs. stvarni salience tier — zaključak za 2 minuta) ilustruje **prednost koju ovaj proposal navodi u uvodu**: eksplicitna pravila čitljiva kao poslovne politike, transparentnost rezonovanja, mogućnost iterativnog menjanja parametara bez velikih refaktora. Ispravka redosleda paljenja je bila dodavanje jednog reda u svako pravilo (`salience 200 / 150`); ispravka backward chaining bug-a bila je lokalna izmena unutar jednog queryja bez dodirivanja ostatka sistema; `pass` fallback je 3-linijska izmena u Javi. Da je sistem bio black-box (npr. trenirana neuronska mreža), nijedan od ovih problema ne bi bio dijagnostikovao posmatranjem tragova, a popravka bi zahtevala ponovni trening ili duboku refaktORIZACIJU.